

Unión Europea

Fondo Social Europeo

El FSE invierte en tu futuro

Laravel

Relación Entre Modelos

Pepe Devesa



GENERALITAT VALENCIANA

Conselleria d'Educació, Investigació, Cultura i Esport



Continguts

1 Relaciones entre modelos **3**
1.1 Relaciones 1:n o one to many 3
1.2 Relaciones n:n o many to many 5
1.3 Uno a Uno (1:1) 8
1.4 Guardar dades relacionades 9

1 Relaciones entre modelos

En esta sesión veremos qué tipos de relaciones se pueden establecer entre los modelos de la aplicación, y cómo se reflejan automáticamente en la base de datos. Para ello, Eloquent permite definir relaciones de varios tipos entre tablas. Éstas se definen a través de los distintos modelos involucrados en la relación, como veremos a continuación.

1.1 Relaciones 1:n o one to many

Para ilustrar esta relación veamos nuestro ejemplo: supongamos que tenemos los modelos `Autor` y `Libro`, de modo que un autor puede tener varios libros, y un libro está asociado a un autor.

La forma de establecer la relación entre ambos consistirá en añadir en la tabla de libros una clave ajena al autor al que pertenece. A la hora de plasmar esta relación en los modelos, usaríamos el método `hasMany`:

```
class Autor extends Model {


    public function libros(){

        return $this->hasMany(Libro::class);

        }

}
```

Se asume que la tabla de libros tiene una clave primaria `id`, y que la clave ajena correspondiente hacia la tabla de autores es `autor_id`. De lo contrario, se pueden especificar otros pasando más parámetros a hasMany.

De este modo obtenemos los libros asociados a un autor:

```
$libros = Autor::findOrFail($id)->libros();
```

Finalmente, también podemos establecer la **relación inversa**, y recuperar el autor al que pertenece un determinado libro, definiendo un método en la clase Libro que emplee `belongsTo`, como en las relaciones uno a uno:

```
class Libro extends Model {

    public function autor(){
        return $this->belongsTo(Autor::class);
    }
```

```
}
```

Y obtener, por ejemplo, el nombre del autor a partir del libro:

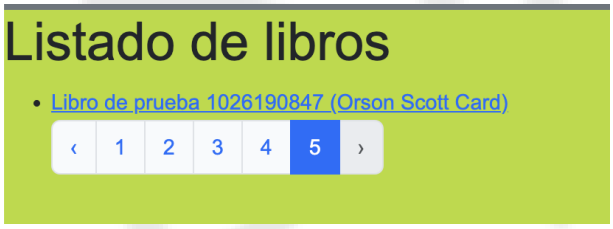
```
$nombreAutor = Libro::findOrFail($id)->autor->nombre;
```

- Para probar cómo funcionan las relaciones, vamos primero a crear un nuevo libro asociado al autor 1. Definimos una ruta de prueba en el archivo `routes/web.php` con este código (deberemos incorporar con use los modelos de Autor y Libro):

```
Route::get('relacionPrueba', function() {
    $autor = Autor::findOrFail(1);
    $libro = new Libro();
    $libro->titulo = "Libro de prueba " . rand();
    $libro->editorial = "Editorial de prueba";
    $libro->precio = 5;
    $libro->autor()->associate($autor);
    $libro->save();
    return redirect()->route('libros.index');
});
```

- Ahora, modificamos la vista `libros/index.blade.php` para que, en el listado, utilice las relaciones entre tablas para mostrar el nombre del autor entre paréntesis junto al título de cada libro:

```
@forelse($libros as $libro)
<li>
    <a href="{{ route('libros.show', $libro) }}">
        {{ $libro->titulo }} ({{ $libro->autor->nombre }})
    </a>
</li>
@empty
    <li>No se encontraron libros</li>
@endforelse
```



(a) Relacion uno a muchos

1.1.1 Acceso eficiente a datos relacionados. *Eager loading*

En el ejemplo anterior, hemos visto cómo, dado un libro, podemos obtener el nombre del autor de este modo en una vista Blade:

```
{{ $libro->autor->nombre }}
```

Sin embargo, este código provoca una nueva consulta en la base de datos para buscar los datos del autor asociado al libro, con lo que, para un listado de 100 libros, estaremos haciendo 100 consultas adicionales para extraer la información de los respectivos autores.

Para evitar esta sobrecarga, podemos emplear una técnica llamada eager loading (que en español podríamos traducir como carga apresurada o impaciente). Consiste en emplear el método `with` para indicar qué relación queremos dejar pre-cargada en el resultado. Por ejemplo, si indicamos algo así en la funció `index` del controlador de libros:

```
public function index()
{
    $libros = Libro::with('autor')->get();
    return view('libros.index', compact('libros'));
}
```

1.2 Relaciones n:n o many to many

Estas relaciones son más difíciles de plasmar, ya que es necesario contar con una tercera tabla que relacione las dos tablas afectadas. Pero vayamos por partes... Para ilustrar este caso, supongamos los modelos `profesor` e `imparte` , de modo que un profesror puede tener impartir varias asignaturas, y una asignatura puede ser impartida porvarios profesores. Definimos un método en el modelo `profesor` que utilice el método `belongsToMany` para indicar con qué otro modelo se relaciona:

```
 class Profesores extends Model
{
    use HasFactory;

    protected $table = 'PROFESORES';

    protected $primaryKey = 'dni';

    public function asignaturas(){
        return $this->belongsToMany(Asignaturas::class, 'IMPARTE','dni','
            asignatura');

    }
}
```

Así ya podremos acceder a los asignaturas de un profesor:

```
public function show(string $id)
{
 $profesor = Profesores::find($id);
 $asignaturas = $profesor->asignaturas;
 o
 $asignaturas= Profesores::findOrFail($id)->asignaturas;

         return view(...); ///y lo enviamos

}
```

En este caso, al otro lado de la relación hacemos lo mismo: definimos un método en el modelo asignatura que indique con `belongsToMany` que puede pertenecer a varios profesores:

```
class Asignaturas extends Model
{
    use HasFactory;
    protected $table = 'ASIGNATURAS';

    protected $primaryKey = 'codigo';

    //Relationships Many to Many
public function profesores(){
    return $this->belongsToMany(Profesores::class, 'imparte','dni','
       asignatura');
}
```

y el resultado:



(b) Listado profesores (c) Listado Asignaturas

(d) Imparten

Figure 1: Listados.

A efectos de automatización, es decir, para que Eloquent establezca los nexos de forma automática, si queremos establecer una relación muchos a muchos entre un modelo `A` y otro `B` , se asume que existirá otra tabla `a_b` (el orden en que se colocan los nombres de las tablas es alfabético), con los campos `a_id` y `b_id` , que relacionen los dos modelos. En nuetro caso, se asumirá que existe una tabla `imparte` con un campo `asignatura` y otro llamado `dni` , que enlacen con los correspondientes id de las tablas de profesor y asignatura. Sdebemos pasar más parámetros a `belongsToMany` para indicarlo.

En el caso de las relaciones muchos a muchos, es posible que nos interese acceder a algún dato de esa tabla intermedia que los relaciona. En ese caso, hacemos uso del atributo `pivot` , predefinido, y que apunta a la tabla o modelo intermedio entre los dos relacionados. Por ejemplo, si quisiéramos obtener la fecha de creación de la relación entre un *profesor* y *asignatura*, podríamos hacer esto:

```
$profesores = asignatura::findOrFail($id)->asignaturas;
for($profesores as $profesor)
{
    echo $profesor->pivot->created_at;
}
```

Pero, antes deberiamos añadir en el modelo:

```
  //Relationships Many to Many
public function profesores(){
    return $this->belongsToMany(Profesores::class, 'imparte','dni','
       asignatura')
    ->withTimestamps();
}
```

1.3 Uno a Uno (1:1)

Suponemos que tenemos dos modelos *Usuario* y *Telefoneo* , de forma que podemos establecer una relación uno en uno entre ellos:

- un usuario tiene un teléfono, y un teléfono pertenece a un usuario.

Para reflejar esta relación en tablas, una de las dos tendría que tener una referencia a la otra. En este caso, podríamos tener un campo usuario_id en la tabla de *telefonos* que indica a quién pertenece este teléfono. Es importante que el campo se diga usuario_id , como veremos a continuación. Para indicar que un usuario tiene un teléfono, añadimos un nuevo método en el modelo de Usuario , que se llame igual que el modelo con el cual queremos conectar (telefoneo , en este caso):

```
class Usuario extends Model
{
    public function telefono()
    {
        return $this->hasOne('App\Models\Telefono');
    }
}
```

Ahora, si queremos obtener el teléfono de un usuario, basta con que hacemos esto:

```
$telefono = Usuario::findOrFail($id)->telefono;
```

Hemos empleado una característica de *Eloquent* denominada propiedades dinámicas, por la cual podemos referenciar un método de relación como sí sería una propiedad (en lugar de usar telefoneo() , hemos empleado telefoneo).

La instrucción anterior obtiene el objeto Telefono asociado con el usuario buscado (a través del **$id del usuario**). Para que esta asociación tenga efecto, hace falta que en la tabla **telefonos** exista un campo **usuario_id** y que se corresponda con un campo **id** de la tabla de **usuarios** , de forma que *Eloquent* establece la conexión entre una y otra mesa.

Tendremos que definir una nueva migración de modificación sobre la tabla telefonos para añadir ese nuevo campo, o refrescar la migración original con él y borrar los contenidos previos. Si queremos

utilizar otros campos diferentes en una y otra tabla para conectarlas, tenemos que indicar dos parámetros más al llamar a *hasOne* . Por ejemplo, así relacionaríamos las dos tablas anteriores, indicando que la clave ajena de *telefonos* es idUsuario , y que la clave local a la cual se referencia en usuarios es codigo :

```
return $this->hasOne('App\Models\Telefono', 'idUsuario', 'codigo');
```

También es posible obtener la relación inversa, es decir, a partir de un teléfono, obtener el usuario al cual pertenece. Para lo cual, añadimos un método en el modelo Telefoneo y empleamos el método **belongsTo** para indicar a qué modelo se asocia:

```
class Telefono extends Model
{
    public function usuario()
    {
        return $this->belongsTo('App\Models\Usuario');
    }
}
```

Nuevamente, podemos especificar otros nombres de clave pasando parámetros adicionales a **belongsTo** , igual que se hace para **hasOne**. De este modo, si queremos obtener el usuario a partir del teléfono, podemos hacerlo así:

```
$usuario = Telefono::findOrFail($idTelefono)->usuario;
```

1.4 Guardar dades relacionades

Suponemos que queremos guardar un usuario con su teléfono asociado. Podemos simplemente guardar el *id* del teléfono como un campo más del usuario:

```
    // Buscamos el teléfono que queremos asociar
    // (suponiendo que existe previamente)
    $telefono = Telefono::findOrFail($idTelefono);
    $usuario = new Usuario();
    $usuario->nombre = "Pepe";
    $usuario->email = "pepe@gmail.com";
    $usuario->telefono_id = $telefono->id;
    $usuario->save();
```

Pero, además, podemos vincular ambos objetos en la relación, usando el método **associate** , de este modo:

```
    // Buscamos el teléfono que queremos asociar
    // (suponiendo que existe previamente)
    $telefono = Telefono::findOrFail($idTelefono);
    $usuario = new Usuario();
```

```
    $usuario->nombre = "Pepe";
    $usuario->email = "pepe@gmail.com";
    $usuario->telefono()->associate($telefono);
    $usuario->save();
```